Q. Solver approach and why it was chosen

I chose a numerical IK solver built around an iterative QP (osqp). At each iteration, Pinocchio computes the current end-effector pose and Jacobian. The solver then asks the solver for a small joint update that reduces Cartesian pose error while respecting bounds. I chose this approach because it is more flexible to condition and enforces joint limits, step limits, and also importantly max jump limit.

Q. How robot-specific kinematics are separated from solver logic

Robot data lives under the "robots" folder. Each robot has its own URDF, dimensions file, and config file. These files describe the robot model.

The Pinocchio wrapper in IK_solver/pinocchio_model.py loads the robot model and provides a common interface. The solver uses the common robot model interface. This keeps the IK logic clean and reusable and keeps robot kinematics out of the optimization code.

Q. How unreachable targets are detected or handled

The solver handles unreachable targets in two stages.

First, it performs an approximate workspace check, using the robot center and radius. If the requested target is outside that workspace sphere, the solver returns "non convergence" with a message explaining that the point is outside the workspace.

Second, for targets that pass the workspace check, the solver iterates until it either reaches tolerance or exhausts the iteration budget. If it cannot reach the target, it returns the best bounded configuration found. Next based on the tolerances configured in the config file, it classifies the best configuration as:

- success: position and orientation errors inside strict tolerances.
- approximate: errors outside strict tolerance but inside the approximate tolerance band.
- non_convergence: errors are outside the approximate tolerance band.

The solver also reports a reason message, when possible, such as what prevented convergence like joint limits, jump limits, or the solution has stagnated.

Q. How pose accuracy is balanced against joint motion and constraints

In the implemented QP solver, the base idea is to minimize cartesian error while regularizing joint motion. This is done in steps; only the orientation errors are scaled; the position errors are used weighed with 1. So, orientation can be made important relative to position. The components of the optimization problem are shared in the OCP.pdf

The solver also includes motion_weight, which discourages unnecessarily large joint updates.

Constraints are handled through bounds:

- joint limits from the robot model
- max_step for each iterative joint update
- max_solution_jump from the starting joint configuration

These constraints mean the solver may return an approximate or non-converged result even when the target is geometrically reachable. That is intentional; a reachable pose may still be rejected if it requires a joint limit violation or a jump that is too large.

Q. Potential issues near singularities

Near a singularity, the Jacobian loses rank. In that situation, small corrections may require very large joint motions.

I planned to use a simple damping to reduce unstable behavior near singularities. Damping makes the update less aggressive. The max_step and max_jump bounds prevent very large joint moves.

These do not fully solve singularity handling. The solver may still converge slowly or choose motions that are mathematically valid but not desirable for a real robot.

Q. What would be required for a physical robot

To make this system safe and useful on a physical robot,

- real joint position, velocity, torque, and state feedback
- velocity, acceleration, jerk, and torque limits
- collision checking against the robot, environment, and self-collisions
- trajectory generation instead of sending isolated IK jumps
- controller-side safety limits and emergency stop behavior
- singularity and workspace boundary monitoring
- rate limiting and filtering of commanded motion

Q. How to extend the system

1. Velocity limits:
Add per-joint velocity limits to config and dimension files. Convert velocity limits into per-iteration step bounds using the controller timestep. These bounds can be added directly to the solver.

2. Collision constraints:
Add a collision model using Pinocchio geometry, hpp-fcl, or another collision library. At each iteration, compute distances to obstacles and self-collision pairs. Convert active distances into linear inequality constraints that prevent motion toward collision. For a first version, collision checking could reject unsafe candidate steps; a stronger version would include collision avoidance in the QP.

3. Redundancy resolution:
For redundant robots, we can add secondary objectives. Like staying near a preferred posture, avoiding joint limits, or minimizing motion. In the QP, these can be represented as additional weighted costs while keeping the cartesian pose task as the primary objective.

4. Dynamics-aware optimization:
Extend the solver from pure kinematics to include mass matrix, Coriolis, gravity, torque limits, and acceleration limits. We can use the full capabilities of Pinocchio to include mass matrix, Coriolis, gravity, torque limits, and acceleration limits. It can compute these dynamics quantities. The optimization would then solve joint velocities, accelerations, or torques while respecting dynamic feasibility. This would move the system closer to a MPC rather than a simple position-level IK.

AI tools Used:

Codex to design parts of the project. Dictation application to write word and text documents. Based on the understanding of the problem statement, ik.md was written. This was used to create a skeleton structure for the code and pipeline.

Once a skeleton of the project was built, each folder/module was tested to expected results.

After the core logic of the solver (solver.py) was implemented, codex was again used to create the demo, meshcat and visualization scripts. Specifically, select values that demonstrate the required case studies. A last-minute unit test was also scripted to test the integrity of the pipeline.